AN IMPROVED UPPER BOUND ON ROMANOFF'S CONSTANT

ANDREW YOO

Abstract. Romanoff's constant is the asymptotic density of the natural numbers that can be expressed as the sum of a prime and a power of two. We lower the upper bound from $\overline{d} < 0.490249407811155$ to $\overline{d} < 0.4901800632900605$, representing an improvement of roughly 6.9×10^{-5}.

1. Introduction

Consider the set of numbers that can be expressed as the sum of a prime (p) and a power of two (2^k, $k \in \mathbb{N}$):

$$\mathcal{R} = \{p + 2^k\}$$

In 1849, de Polignac conjectured that $\mathcal{R}$ spanned every odd number greater than 3. Upon realizing that his conjecture was false (the first counterexamples occur at $127, 149, \ldots$), he turned his attention toward the set's density:

$$d(n) = \frac{1}{n} \#\left(\mathcal{R} \cap [1, n]\right)$$

Let $\underline{d} = \liminf d(n)$ and $\overline{d} = \limsup d(n)$. It is conjectured that $\underline{d} = \overline{d} = d$.

The first nontrivial result arrived when Romanoff [Rom34] proved that $\underline{d} > 0$. Since then, the lower bound has been increased substantially by [CS04], [HR06], [Pin06], and [HS10]. Elsholtz and Schlage-Puchta [ES18] hold the current record of $\underline{d} > 0.107648$.

Regarding the upper bound, Erdős [Erd50] achieved the first nontrivial bound when he proved that $\overline{d} < 0.5$ using an explicit arithmetic progression. Habsieger and Roblot [HR06] improved the bound to $\overline{d} < 0.490941$ by proving an algorithm for computing $\overline{d}$ based on an inputted squarefree number. Chen, Dai, and Li [CDL24] introduced parallelism and GPU-acceleration, resulting in $\overline{d} < 0.49034109$. Finally, in a recent unpublished work, Griego [Gri26] introduced a new application of the Chinese Remainder Theorem, attaining $\overline{d} < 0.49024941$.

Despite significant efforts (tracked in [Dav+26]), the gap between $\underline{d}$ and $\overline{d}$ remains large. The upper bound has remained particularly elusive, with increasingly modest improvements over the trivial bound. However, the current upper bound is likely sharper than the current lower bound: [Rom83] conjectured that $d \approx 0.434$. [Del+20], strengthening the methods, conjectured that $d \approx 0.437$ if it indeed exists.

In this paper, we prove the CRT approach employed by [Gri26]. The resulting algorithm is then verified by reproducing the results of [CDL24]. Finally, we show

Date: August 6, 2026.

2020 *Mathematics Subject Classification.* 11P32 (primary); 11Y60 (secondary).

that the selection of both seeds and clusters is nontrivial; using a simple optimization heuristic, we select a more optimized prime set and hence improve the upper bound.

Theorem 1. *The upper density satisfies* $\overline{d} < 0.4901800632900605$.

Section 2 contains a description and proof of the CRT algorithm, reproduction of [CDL24], and a proof of Theorem 1 using a 38-prime set. Section 3 comments on the methodology used in both the seed search and cluster search. Section 4 includes some final remarks. Section 5 contains a short AI use disclosure.

2. Theoretical Framework

2.1. **The Habsieger-Roblot Theorem.** We refer readers to [HR06] and [CDL24] for a detailed overview, but we provide a very brief summary here.

Let $\mathcal{P}$ be a finite set of odd primes, $M = \prod_{p\in\mathcal{P}} p$, and $T = \mathrm{ord}_M(2)$. Also, let

$$f(a) = \left\{\overline{k} \in \mathbb{Z}/T\mathbb{Z} : a - 2^{\overline{k}} \in (\mathbb{Z}/M\mathbb{Z})^*\right\}$$
$$\delta(\nu) = \{a \in \mathbb{Z}/M\mathbb{Z} : \#f(a) = \nu\}$$

for a a residue class modulo M and $0 \le \nu \le T$. Then we recall the following from [HR06]. [1]

Lemma 1. $\overline{d}$ *is restricted by*

$$\overline{d} \le \sum_{\nu=0}^{T} \delta(\nu) \cdot \min\left(\frac{1}{2M}, \frac{\nu}{T \cdot \varphi(M) \cdot \log(2)}\right)$$

As $\#\mathcal{P}$ grows large, M increases exponentially, and the computation of f, whose image contains 2^T subsets of $\mathbb{Z}/T\mathbb{Z}$, becomes extremely expensive.

Since f takes few values in practice, the alternative formulation of δ allows for much faster computation through induction upon $\mathcal{P}$:

$$g(I) = \{a \in \mathbb{Z}/M\mathbb{Z} : f(a) = I\}$$
$$G(I) = \#g(I),\ I \subset \mathbb{Z}/T\mathbb{Z}$$
$$\delta(\nu) = \sum_{\#I=\nu} G(I)$$

Further computational improvements were introduced by [CDL24], enabling parallelism and GPU-acceleration, though they are outside the scope of this paper.

2.2. **The Chinese Remainder Theorem Approach.** A different approach, suggested by [Gri26], involves exploiting the Chinese Remainder Theorem to break up the computation of the Habsieger-Roblot bound into individual components.

[1] Our Lemma 1 differs from [HR06]'s Lemma 2 by a factor of 2. [HR06] define their density with denominator $x/2$ (i.e., among *odd* naturals), while we consider the density among *all* naturals. [CDL24] use the same convention as this paper in their calculations, but the formula printed in their Lemma 4.1 retains the unadjusted form from [HR06].

2.2.1. *CRT Partition and Decoupling.* Partition the prime set $\mathcal{P} = \mathcal{P}_0 \cup \mathcal{P}_1 \cup \cdots \cup \mathcal{P}_m$ subject to the following conditions:

(1) The *seed set* $\mathcal{P}_0$ is the set of primes processed jointly via the full Habsieger-Roblot dynamic program;
(2) The *cluster* $\mathcal{P}_i$ $(i \geq 1)$ is a set of primes sharing a common order $d_i = \operatorname{ord}_p(2)$;
(3) the cluster orders $d_1, \ldots, d_m$ are pairwise coprime;
(4) each d_i is coprime to $T_0 = \operatorname{lcm}_{p \in \mathcal{P}_0} \operatorname{ord}_p(2)$.

Define M_0 and M_i (for $i \geq 1$).

$$M_0 = \prod_{p \in \mathcal{P}_0} p$$
$$M_i = \prod_{p \in \mathcal{P}_i} p$$
$$M = M_0 \prod_{i=1}^{m} M_i$$

Since the cluster orders are pairwise coprime and each is coprime to T_0, T can be factored as follows.

$$T = T_0 \prod_{i=1}^{m} d_i$$

Lemma 2 (CRT decoupling)**.** *Define $\nu_0(a_0)$ for the seed and h_i for each cluster $i \geq 1$.*

$$\nu_0(a_0) = \#\{r_0 \bmod T_0 : \gcd(a_0 - 2^{r_0}, M_0) = 1\}$$
$$h_i(a_i) = \#\{r_i \bmod d_i : \gcd(a_i - 2^{r_i}, M_i) = 1\}$$

Then, for every full residue $a = (a_0, \ldots a_m) \bmod M$,

$$\nu(a) = \nu_0(a_0) \prod_{i=1}^{m} h_i(a_i)$$

Proof. We proceed in three stages.

CRT Decomposition. Since $T_0, d_1, \ldots, d_m$ are pairwise coprime and their product equals T, the CRT provides the ring isomorphism

$$\mathbb{Z}/T\mathbb{Z} \;\cong\; \mathbb{Z}/T_0\mathbb{Z} \times \mathbb{Z}/d_1\mathbb{Z} \times \cdots \times \mathbb{Z}/d_m\mathbb{Z}$$

Consequently, every exponent class $r \bmod T$ corresponds uniquely to a tuple

$$r \longleftrightarrow (r_0 \bmod T_0, r_1 \bmod d_1, \ldots, r_m \bmod d_m)$$

whose components vary independently over their respective residue rings. The same reasoning applied to M_i, which are pairwise coprime by construction, gives the analogous decomposition under the Chinese Remainder Theorem

$$a \bmod M \longleftrightarrow (a_0 \bmod M_0, a_1 \bmod M_1, \ldots, a_m \bmod M_m)$$

where a_i denotes the residue of a modulo M_i.

Admissibility. Since M is squarefree,

$$\gcd(a - 2^r, M) = 1 \iff \gcd(a - 2^r, p) = 1 \ \forall p \in \mathcal{P}$$

For seed primes, take $p \in \mathcal{P}_0$ and let $d = \mathrm{ord}_p(2)$. By definition of d and T_0, $d \mid T_0$. So $r \equiv r_0 \bmod T_0 \implies r \equiv r_0 \bmod d \implies 2^r \equiv 2^{r_0} \bmod p$. Hence

$$\gcd(a - 2^r, p) = 1 \iff \gcd(a_0 - 2^{r_0}, p) = 1$$

Requiring this for all $p \in \mathcal{P}_0$ simultaneously is equivalent to $\gcd(a_0 - 2^{r_0}, M_0) = 1$. Therefore, the set of admissible seed-level exponent classes is

$$f_0(a_0) = \{r_0 \bmod T_0 : \gcd(a_0 - 2^{r_0}, M_0) = 1\}$$

which, by definition, has the cardinality $\#f_0(a_0) = \nu_0(a_0)$.

Cluster primes are admitted similarly. For $p \in \mathcal{P}_i$ with order d_i, the reduction $r \equiv r_i \bmod d_i \implies 2^r \equiv 2^{r_i}$, so

$$\gcd(a - 2^r, p) = 1 \iff \gcd(a_i - 2^{r_i}, p) = 1$$

Requiring this for all $p \in \mathcal{P}_i$ is equivalent to $\gcd(a_i - 2^{r_i}, M_i) = 1$. Thus, the admissible set for cluster i is:

$$f_i(a_i) = \{r_i \bmod d_i : \gcd(a_i - 2^{r_i}, M_i) = 1\}$$

which has the cardinality $\#f_i(a_i) = h_i(a_i)$.

Product. An exponent class $r \equiv (r_0, \ldots, r_m)$ is admissible for the full residue when it is admissible for all $p \in \mathcal{P}$. From the above, this occurs when r_0 and r_i are admissible for the seed and each cluster, respectively.

$$r \in f(a) \iff [r_0 \in f_0(a_0)] \wedge [r_i \in f_i(a_i)]$$

Because the components of $(r_0, r_1, \ldots r_m)$ vary independently, the set $f(a)$ is the Cartesian product of the individual admissible sets.

$$\begin{aligned} f(a) &= f_0(a_0) \times f_1(a_1) \times \cdots \times f_m(a_m) \\ \nu(a) &= \#f(a) \\ &= \#f_0(a_0) \prod_{i=1}^{m} \#f_i(a_i) \\ &= \nu_0(a_0) \prod_{i=1}^{m} h_i(a_i) \end{aligned}$$

□

2.2.2. *Cluster Polynomials.* For each cluster i the distribution of the multiplier $h_i(a_i)$ across residues $a_i \bmod M_i$ is recorded by the cluster's *cluster polynomial*

$$C_i(x) = \sum_h c_{i,h} x^h$$
$$c_{i,h} = \#\{a_i \bmod M_i : h_i(a_i) = h\}$$

Lemma 3 gives an explicit formula for single-prime clusters, while Algorithm 1 handles multi-prime clusters in $\mathcal{O}(kd)$ time.

Lemma 3 (Single-prime cluster polynomials)**.** *For a cluster consisting of a single prime p with order d:*

$$C_p(x) = dx^{d-1} + (p-d)x^d$$

Proof. By definition $d = \mathrm{ord}_p(2)$. The powers $2^0, 2^1, \ldots 2^{d-1}$ are distinct modulo p and constitute the full set of residues that are congruent to a power of 2 modulo p.

For some residue $a_i \bmod p$, there are two cases. If a_i is a power residue, then $a_i \equiv 2^j \bmod p$ for a unique $j \in \{0, 1, \ldots, d-1\}$. The exponent class $r \equiv j \bmod d$ is blocked because $a_i - 2^r \equiv 0 \bmod p \implies \gcd(a_i - 2^r, p) \neq 1$. The remaining $d-1$ exponent classes are unblocked, giving $h_i(a_i) = d-1$. There is one residue for each j.

If a_i is not a power residue, then $a_i \not\equiv 2^r \bmod p$ for every $r \bmod d$, so no class is blocked and $h_i(a_i) = d$. There are $p-d$ such residues.

Summing both cases yields

$$C_p(x) = dx^{d-1} + (p-d)x^d$$

□

For a cluster containing multiple primes $p_1, \ldots, p_k$, an exponent class $r \bmod d$ is admissible if and only if it avoids every prime in the cluster. Define the *bad set*

$$B = \{r \bmod d : a_{ij} \equiv 2^r \bmod p_j \ \exists j\}$$

and let $s = \#B$. Then $h_i(a_i) = d - s$ since exactly $d-s$ of the d exponent classes avoid all primes in the cluster. Each prime contributes at most one element to B. If the residue a_{ij} is a power residue $2^r \bmod p_j$, it adds $\{r\}$ to B. Otherwise, it adds nothing. Consequently, $0 \leq s \leq \min(k, d)$.

For large primes, the naive approach is impractical. Algorithm 1 avoids this by aggregating residues with the same value of s, processing primes individually, and tracking the distribution of s across the tuples already seen.

The algorithm is justified by induction. Before processing, $s = 0$, matching the initial dist. There are three possible cases:

(1) Non-power residue: $p - d$ choices, state remains s.
(2) Power residue (already blocked): s choices, state remains s.
(3) Power residue (new): $d - s$ choices, state increases by 1.

Since the state space never exceeds $d + 1$ entries, k iterations cost $\mathcal{O}(kd)$.

Algorithm 1 Multi-prime cluster polynomials

Require: Primes $p_1, \ldots, p_k$ such that $d = \mathrm{ord}_p(2)$ for all p
Ensure: Polynomial $C(x) = \sum_h c_h\ x^h$
  dist $\leftarrow \{0 \mapsto 1\}$
  for $p \in \{p_1, \ldots, p_k\}$ **do**
    next $\leftarrow \varnothing$
    for each $(s, wt) \in$ dist **do**
      next$[s] \leftarrow$ next$[s] + wt \cdot (p - d)$ ▷ non-power
      if $s > 0$ **then**
        next$[s] \leftarrow$ next$[s] + wt \cdot s$ ▷ power (already blocked)
      end if
      if $s < d$ **then**
        next$[s+1] \leftarrow$ next$[s+1] + wt \cdot (d - s)$ ▷ power (new)
      end if
    end for
    dist $\leftarrow$ next
  end for
  return $\{(d - s) \mapsto \mathrm{dist}[s] \mid s = 0, \ldots, \min(k, d)\}$

2.2.3. *Multiplicative Convolution.* By Lemma 2, the total admissible count factors as a product of per-cluster multipliers. Therefore, the global distribution of the multiplier product $h = \prod_{i=1}^{m} h_i$ is the multiplicative convolution of the cluster polynomials:

$$\mathcal{D}(x) = \sum_h \mathcal{D}_h x^h$$
$$\mathcal{D}_h = \sum_{h_1 \ldots h_m = h} c_{1,h_1} \ldots c_{m,h_m}$$

Each $\mathcal{D}_h$ counts the number of residue tuples $(a_1, \ldots, a_m)$ whose multiplier product equals h. The total weight satisfies

$$\sum_h \mathcal{D}_h = \prod_{i=1}^{m} M_i = \frac{M}{M_0}$$

which serves as an internal consistency check: the convolution must account for every residue tuple in the cluster product.

2.2.4. *CRT Bound.* Consider the seed histogram

$$\delta_0(s) = \#\{a_0 \bmod M_0 : \nu_0(a_0) = s\}$$

which records how many seed residues a_0 achieve each value of $\nu_0(a_0)$. This histogram is computed by running the full Habsieger-Roblot algorithm on $\mathcal{P}_0$. This is feasible when M_0 is kept small. By Lemma 2, exactly $\delta_0(s)\mathcal{D}_h$ full residues $a \bmod M$ satisfy $\nu_0(a_0) = s$ and $\prod_{i=1}^{m} h_i(a_i) = h$, hence $\nu(a) = sh$.

Theorem 2 (CRT Bound)**.** *Under the partition described above, the following is a refinement of Lemma 1.*

$$\overline{d} \leq \sum_{s} \delta_0(s) \sum_{h} \mathcal{D}_h \min\left(\frac{1}{2M}, \frac{sh}{T \cdot \varphi(M) \cdot \log(2)}\right)$$

Proof. Recall the bound from Lemma 1:

$$\overline{d} \leq \sum_{t=0}^{T} \delta(t) \min\left(\frac{1}{2M}, \frac{t}{T \cdot \varphi(M) \cdot \log(2)}\right)$$

where $\delta(t) = \#\{a \bmod M : \nu(a) = t\}$ counts the residues attaining t admissible exponent classes.

By Lemma 2, each full residue $a = (a_0, a_1, \ldots, a_m)$ satisfies

$$\nu(a) = \nu_0(a_0) \prod_{i=1}^{m} h_i(a_i)$$

Let s range over the admissible counts and let h range over the multiplier products in $\mathcal{D}$. A residue with seed component s and cluster product h has full admissible count sh. There are $\delta_0 \mathcal{D}_h$ such residues, so

$$\delta(t) = \sum_{sh=t} \delta_0(s) \mathcal{D}_h$$

where the sum runs over pairs (s, h) whose product equals t. Substituting yields

$$\begin{aligned}
\overline{d} &\leq \sum_{t=0}^{T} \delta(t) \min\left(\frac{1}{2M}, \frac{t}{T \cdot \varphi(M) \cdot \log(2)}\right) \\
\overline{d} &\leq \sum_{t=0}^{T} \left[\sum_{sh=t} \delta_0(s) \mathcal{D}_h\right] \min\left(\frac{1}{2M}, \frac{t}{T \cdot \varphi(M) \cdot \log(2)}\right) \\
\overline{d} &\leq \sum_{s} \delta_0(s) \sum_{h} \mathcal{D}_h \min\left(\frac{1}{2M}, \frac{sh}{T \cdot \varphi(M) \cdot \log(2)}\right)
\end{aligned}$$

□

2.3. **Comparison of Methods.** To verify the CRT algorithm, 28 test cases were taken from [CDL24] and reproduced. Table 1 contains a summary of the results, listed in order of appearance. The "errors" are due to the use of floating-point arithmetic in [CDL24]; our program, which uses exact (rational) arithmetic, is immune. [2]

[2]Inconsistencies in reported precision, which varied from 9–15 decimal places, also may have inflated the resulting errors.

$\mathcal{P}$	M	Error
$\{3\}$	3	0
$\{3,5\}$	15	0
$\{3,5,7\}$	105	0
$\{3,5,7,11\}$	1155	1.37e−9
$\{3,5,7,11,13\}$	15015	1.01e−9
$\{3,5,7,11,13,17\}$	255255	4.48e−9
$\{3,5,7,11,13,17,19\}$	4849845	6.46e−10
$\{3,5,7,11,13,17,19,31\}$	150345195	6.05e−10
$\{3,5,7,11,13,17,19,31,41\}$	6164152995	4.08e−10
$\{3,5,7,11,13,17,19,31,41,73\}$	449983168635	1.03e−9
$\{3,5,7,11,13,17,19,31,41,73,241\}$	108445943641035	4.97e−9
$\{3,5,7,11,13,17,19,31,41,73,241,257\}$	27870607515745995	5.63e−10
$\{3,5,7,11,13,17,19,31,37,41,61,73,241\}$	244762494797815995	9.77e−11
$\{3,5,7,11,13,17,19,29,31,37,41,61,73\}$	29452748336666655	3.28e−9
$\{3,5,7,11,13,17,19,31,37,41,53,61,73\}$	53827436615287335	2.98e−10
$\{3,5,7,11,13,17,19,23,31,37,41,61,73\}$	23359076267011485	1.60e−9
$\{3,5,7,11,13,17,19,23,29,31,37,41,61,73\}$	677413211743333065	3.33e−16
$\{3,5,7,11,13,17,19,23,29,31,37,41\}$	152125131763605	2.06e−9
$\{3,5,7,11,13,17\}$	255255	2.01e−15
$\{3,5,7,13,17,241\}$	5592405	4.42e−15
$\{3,5,7,11,13,31,41,61,151,331,1321\}$	76861433640456465	3.89e−15
$\{3,5,7,11,13,17,19,31,41,73,241\}$	108445943641035	3.62e−15
$\{3,5,7,11,17,19\}$	373065	3.33e−16
$\{3,5,7,11,17,23\}$	451605	3.39e−16
$\{3,5,7,11,17,29\}$	569415	7.22e−16
$\{3,5,7,11,17,19,23\}$	8580495	4.19e−16
$\{3,5,7,11,17,19,29\}$	10818885	9.40e−16
$\{3,5,7,11,17,23,29\}$	13096545	4.01e−16

TABLE 1. Validation of CRT algorithm against results from [CDL24].

This also provided an opportunity to benchmark our program's performance. Table 2 compares the performance on the set with $M = 27870607515745995$. Table 3 compares the performance on the set with $M = 677413211743333065$. On both benchmarks, the CRT approach demonstrates superior performance.

Source	Hardware	Runtime
[HR06]	Intel Xeon 2.4GHz (2.1 GB)	35 min
[CDL24]	Intel i9-13980HX and Nvidia RTX4070 (16 GB)	3 min
This Work	Intel Core 5 210H (16 GB)	1 s

TABLE 2. Benchmark results for $M = 27870607515745995$

Source	Hardware	Runtime
[CDL24]	2× Intel Xeon E5-2697v2 and Nvidia V100 (1536 GB)	167 h
This Work	Intel Core 5 210H (16 GB)	15 s

TABLE 3. Benchmark results for $M = 677413211743333065$

2.4. **Proof of Theorem 1.** The proof of Theorem 1 consists of enumerating a set of 38 primes and computing the associated $\overline{d}$.

Proof. Consider the following set of primes.

$$\begin{aligned}\mathcal{P} = \{&3,\ 5,\ 7,\ 11,\ 13,\ 17,\ 19,\ 23,\ 29,\ 31,\ 37,\ 41,\ 47,\ 61,\ 73,\ 89,\ 167,\ 223,\}\\ &233,\ 257,\ 263,\ 359,\ 383,\ 431,\ 439,\ 479,\ 503,\ 1103,\ 1399,\ 1433,\ 2089,\\ &2351,\ 2687,\ 3391,\ 4513,\ 6361,\ 8191,\ 9719\}\end{aligned}$$

We now give a list of parameters for reproduction.
Partition $\mathcal{P}$ into a 14-prime seed set and 18 clusters. $\mathcal{P}_0$ and T are given below.

$$\begin{aligned}\mathcal{P}_0 &= \{3, 5, 7, 11, 13, 17, 19, 29, 31, 37, 41, 61, 73, 257\}\\ T_0 &= \operatorname{lcm}(2, 4, 3, 10, 12, 8, 18, 28, 5, 36, 20, 60, 9, 16)\\ &= 5040\end{aligned}$$

A table with the polynomials for each cluster can be found in Appendix A.
We obtain the following constants, which are used to compute the final bound

$$\begin{aligned}T &= 6\ 451\ 014\ 820\ 202\ 817\ 249\ 039\ 156\ 241\ 677\ 297\ 360\\ M &= 2\ 355\ 707\ 395\ 587\ 467\ 342\ 932\ 731\ 100\ 136\ 780\ 153\ 718\ 076\\ &\quad\ 411\ 525\ 881\ 805\ 244\ 678\ 633\ 806\ 008\ 753\ 031\ 518\ 992\ 062\ 935\\ \varphi(M) &= 615\ 938\ 575\ 607\ 136\ 358\ 908\ 423\ 957\ 635\ 764\ 550\ 811\ 567\\ &\quad\ 597\ 607\ 384\ 847\ 097\ 592\ 328\ 250\ 744\ 499\ 077\ 120\ 000\ 000\ 000\end{aligned}$$

We must now plug in an exact value for $\log(2)$. We use a lower bound on the value of $\log(2)$, which is equivalent to an upper bound on its reciprocal. The following is a valid approximation (for finite n). This follows from the fact that $\log(2) = 2\operatorname{arctanh}(\frac{1}{3})$.

$$\log(2) > \sum_{j=0}^{n} \frac{2}{(2j+1)(3^{2j+1})}$$

Our computation uses the first 35 terms—enough to accommodate quadruple-precision—though this is likely overkill.

Using this approach, we obtain the bound $\overline{d} \leq 0.4901800632900605$. See Appendix A for the exact numerator and denominator.

□

3. Search Methodology

In this section, we discuss the optimization heuristics used for prime selection.

3.1. **Cluster Search.**

3.1.1. *Candidate Generation.* For a candidate prime p with order d, we have that $d \mid (p-1)$, so primes take the form $p = 2kd + 1$ for some $k \geq 1$. We precompute a simple sieve, serving as a bound on the primes considered.

Two types of candidate primes are generated from the cluster set $\mathcal{P}_i$.

(1) **New Order**: d must satisfy $\gcd(d, T_0) = 1$ and $\gcd(d, d') = 1$ for all existing cluster orders d'.

(2) **Old Order**: p is added to the existing cluster $\mathcal{P}_i$ with d_i.

For each candidate prime, the bound is updated efficiently:

(1) $M' = M \cdot p, \quad \varphi(M') = (p-1)\varphi(M)$
(2) If d is new: extend the multiplier distribution $\mathcal{D}$ via merge convolution with the single-prime polynomial $C_p(x) = d\,x^{d-1} + (p-d)\,x^d$ in $O(|\mathcal{D}|)$ time.
(3) If d already exists: recompute the cluster polynomial for $\mathcal{P}_i \cup \{p\}$ via Algorithm 1, then replace the old cluster's contribution in the multiplier distribution.
(4) Recompute the sum in Theorem 2 using prefix-sum binary search over the sorted multiplier distribution.

3.1.2. *Beam Search.* The bound provided by Theorem 2 is monotonic with respect to M. That being said, the *rate* of improvement can be improved dramatically by careful selection of primes. An expensive fitness function makes the naive approach infeasible, especially with modest compute.

Due to the relative smoothness of the landscape, a simple greedy search is attractive. However, greedy search suffers from a critical shortcoming. It rarely allocates primes to existing clusters. As a result, we use beam search, which maintains width w candidates simultaneously.

As an initial test, the beam search algorithm was run on the same seed set as [Gri26]. It achieved an improvement of roughly 4.8×10^{-7} for the set with the following M:

$$M = 5171010293623226768764553681881267803263479$$
$$7481190332081064550893712639942412174040 65$$

3.2. **Seed Search.** The seed $\mathcal{P}_0$ is the most important subset of any given $\mathcal{P}$, and therefore has the potential for the greatest improvement. The seed set directly determines the two important constants M_0 and T_0. Two opposing objectives must be balanced:

- **Seed strength**. Every additional prime is multiplied by M_0 and reduces the seed-only bound (Theorem 2 with empty clusters).
- **Cluster availability**. Every cluster must have an order coprime to T_0. Smaller T_0 eliminates fewer cluster orders.

As a result, the goal of the seed search is to maximize M_0 while minimizing the number of unique prime factors comprising T_0.

Seed sets with plausible histogram sizes and cluster search spaces, namely those with $T_0 \in \{360, 504, 840, 2520, 5040, 10080, 20160\}$, were tested. Ultimately, $T_0 = 5040$ was chosen, as adding the prime 257 does not remove any more clusters, and the histogram computation remained reasonable.

Algorithm 2 Beam search for cluster generation

Require: Seed histogram δ_0; beam width w; max rounds R
Ensure: Best cluster set found
  $S \leftarrow$ {empty cluster set}
  for $r = 1$ to R **do**
    $C \leftarrow \varnothing$
    for each state $s \in S$ **do**
      for each new-order candidate (p, d) from s **do**
        Evaluate bound; add (p, d, bound) to C
      end for
      if s has existing clusters **then**
        for each existing-order candidate (p, d) from s **do**
          Evaluate bound; add (p, d, bound) to C
        end for
      end if
    end for
    Sort C by bound (ascending)
    Apply the top w candidates from C to form new states, skipping duplicate (s, p, d) transitions
    $S \leftarrow$ the resulting states
  end for
  return the best state in S

4. Final Remarks

As all computations in this paper were performed on a laptop with an Intel Core 5 210H processor (8 cores, 12 threads) and 16 GB of RAM, the bound achieved is likely nowhere near optimal. The embarrassingly parallel nature means that a distributed search over many cores could significantly reduce search times and make larger seed sets plausible.

However, the largest improvements still come from the seed set, whose bound is still limited by the methods from [HR06]. As a result, we believe that a significant theoretical improvement will be needed if the upper bound is to approach the $d \approx 0.437$ conjectured in [Del+20].

5. AI Use Disclosure

The author acknowledges the contributions of DeepSeek V4 Pro and DeepSeek V4 Flash, especially regarding code. All code was verified by the author. This manuscript was entirely written by the author, who takes full responsibility for its content.

Appendix A. Details for the proof of Theorem 1

Table 4 contains a list of clusters and their respective cluster polynomials.

d	$C(x)$
11	$110x^9 + 1001x^{10} + 936x^{11}$
13	$13x^{12} + 8178x^{13}$
23	$23x^{22} + 24x^{23}$
29	$21924x^{26} + 2712892x^{27} + 82798335x^{28} + 451337760x^{29}$
37	$37x^{36} + 186x^{37}$
43	$1806x^{41} + 432795x^{42} + 3754288x^{43}$
47	$2162x^{45} + 318237x^{46} + 10289664x^{47}$
53	$53x^{52} + 6308x^{53}$
73	$73x^{72} + 366x^{73}$
79	$79x^{78} + 2608x^{79}$
83	$83x^{82} + 84x^{83}$
113	$113x^{112} + 3278x^{113}$
131	$131x^{130} + 132x^{131}$
179	$31862x^{177} + 256865x^{178} + 225720x^{179}$
191	$191x^{190} + 192x^{191}$
233	$233x^{232} + 1166x^{233}$
239	$239x^{238} + 240x^{239}$
251	$251x^{250} + 252x^{251}$

TABLE 4. A list of clusters and their respective polynomials.

Let $\overline{d} \leq \frac{N}{D} \approx 0.4901800632900605$. Exact values of N and D are below.

$$N = 729847810508355757538550989390443411056085082775212684452845317$$
$$807685858089951125130287100217894836461668211797635767573876511$$
$$127776584592073238245366200691072627820807153721633017122829919$$
$$010224378700364396855352551208902981616560019201119878275925545$$
$$016925177774080000$$

$$D = 148893817836992213395316970583294875683936302054058164790833824$$
$$291515684196990865258312960197894290909283384615773478223039675$$
$$630978396395362980452451070901412371084627511134335564347229221$$
$$807551414985371682665667793046253824724747736119116220030552755$$
$$2999424000000000000$$

REFERENCES

[CDL24] Yuda Chen, Xiangjun Dai, and Huixi Li. "Some results on a conjecture of de Polignac about numbers of the form $p + 2^k$". In: (2024). DOI: 10.48550/arXiv.2402.06644. arXiv: 2402.06644 [math.NT]. URL: https://arxiv.org/abs/2402.06644.

[CS04] Yong-Gao Chen and Xue-Gong Sun. "On Romanoff's constant". In: *Journal of Number Theory* 106.2 (2004), pp. 275–284. ISSN: 0022-314X. DOI: https://doi.org/10.1016/j.jnt.2003.11.009. URL: https://www.sciencedirect.com/science/article/pii/S0022314X03002191.

[Dav+26] Damek Davis et al. *Optimization Constants in Mathematics*. GitHub repository. 2026. URL: https://github.com/teorth/optimizationproblems.

[Del+20] Gianna Del Corso et al. "On Computing the density of integers of the form $2^n + p$". In: *Mathematics of Computation* 89.325 (2020).

[Erd50] Paul Erdős. "On integers of the form $2^k + p$ and some related problems". In: *Summa Brasilienis Mathematicae* 2 (1950), pp. 113–123.

[ES18] Christian Elsholtz and Jan-Christoph Schlage-Puchta. "On Romanov's constant". In: *Mathematische Zeitschrift* 288 (2018), pp. 713–724. DOI: 10.1007/s00209-017-1908-x.

[Gri26] Sebastian Griego. *C45 Romanoff upper-density certificate*. GitHub repository. 2026. URL: https://github.com/sebastian-griego/c45-romanoff-certificate.

[HR06] Laurent Habsieger and Xavier-Francois Roblot. "On Integers of the form $p + 2^k$". In: (2006). DOI: 10.4064/aa122-1-1. HAL: 00863145.

[HS10] Laurent Habsieger and Jimena Sivak-Fischler. "An effective version of the Bombieri-Vinogradov theorem, and applications to Chen's theorem and to sums of primes and powers of two". In: *Archiv der Mathematik* 95 (2010), pp. 557–566. DOI: 10.1007/s00013-010-0202-5.

[Pin06] J. Pintz. "A note on Romanov's constant." In: *Acta Mathematica Hungarica* 112.1-2 (2006), pp. 1–14.

[Rom34] N.P. Romanoff. "Über einige Sätze der additiven Zahlentheorie". In: *Mathematische Annalen* 109 (1934), pp. 668–678. URL: http://eudml.org/doc/159702.

[Rom83] Francesco Romani. "Computations concerning primes and powers of two". In: *Calcolo* 20.3 (1983), pp. 319–336.